ASSIGNMENT – 1

CSA16 – DATA WAREHOUSING AND DATA MINING

Student Name: M. Pranay

Register Number: 192525382

Department: Computer Science and Engineering

TASK 1 – ANALYSE THE DECISION-SUPPORT REQUIREMENT (4 Marks)

1.1 Business questions, classification, measures and dimensions

Business question	Type	Measures	Required dimensions
1. Which product categories show declining net sales quarter over quarter in each region?	Strategic	Net sales, sales growth %	Time (quarter/year), Product (category/department), Store (region)
2. Which promotions increased average basket value, and in which stores?	Tactical	Average basket value, discount, net sales, order count	Promotion, Store, Date, Customer segment
3. How does weekend sales performance differ from weekday performance for each department?	Tactical	Gross sales, net sales, quantity sold, order count	Date (day type/week), Product (department), Store
4. Which customer segments return the most items, and of which products?	Strategic	Returned quantity, return value, return rate	Customer segment, Product, Date, Store
5. What is each store's	Strategic	YTD net sales,	Date, Store, Region

year-to-date sales compared with the same period last year?		previous-year YTD sales, growth %	
6. Which stores have the highest daily sales and lowest sales?	Tactical	Daily net sales, order count, average basket value	Date, Store, City, Region
7. Which products frequently sell together?	Strategic	Co-purchase count, basket count	Product, Category, Store, Date, Promotion
8. Which suppliers provide products with high sales and low return rates?	Strategic	Sales quantity/value, return quantity/rate	Supplier, Product, Category, Date, Store

1.2 OLTP system versus decision-support system

Aspect	**FreshCart OLTP**	**FreshCart decision-support warehouse**
Purpose	Records daily billing, orders, returns and customer transactions.	Analyses historical performance and supports management decisions.
Data currency	Current operational data; updated continuously.	Historical and integrated data; refreshed on a planned schedule, initially daily/hourly as required.
Structure	Normalised tables such as Orders, OrderLines, Customers and Products.	Dimensional star schema with fact and dimension tables.
Query pattern	Short inserts, updates and lookups for individual transactions.	Long-running aggregations, comparisons, trends and multidimensional queries.

Users	Cashiers, store staff and operational applications.	Managers, analysts and executives.
Volume	Large transaction count, optimised for transaction processing.	Historical data across all stores and channels, optimised for analysis.

FreshCart's OLTP database is unsuitable for several analytical questions because they require joining many operational tables and aggregating millions of order lines over long periods. Such queries compete with billing transactions and slow down counters during peak hours. Separate departments also apply different net-sales calculations. A warehouse solves this by integrating data, applying one approved calculation rule, preserving history and presenting a consistent dimensional structure.

Questions 1, 4, 5, 7 and 8 are especially inefficient on OLTP because they require historical comparisons, multi-dimensional aggregation, return analysis or product co-purchase analysis. The warehouse should support consistent net-sales reporting, historical trends, drill-down by time/product/store/customer, and scheduled dashboard refreshes.

TASK 2 – PROPOSE THE WAREHOUSE ARCHITECTURE (3 Marks)

Recommended architecture: an enterprise data warehouse with dependent data marts. This gives FreshCart one integrated and governed source of truth while allowing Sales, Returns and Promotion teams to use subject-specific marts.

OLTP sources (Orders, OrderLines, Customers, Products, Stores, Promotions, Returns)

|

Extract / CDC

|

STAGING AREA

(raw copies, source identifiers, load timestamp)

|

ETL: clean, standardise, validate,

deduplicate, conform dimensions

|

ENTERPRISE DATA WAREHOUSE

(Sales fact, Returns fact, dimensions)

|

METADATA REPOSITORY Aggregate tables/indexes

|

DEPENDENT DATA MARTS

Sales | Returns | Promotions

|

OLAP SERVER

(ROLAP / SQL aggregation layer)

|

Metabase / Superset / Python reports

dashboards, queries, exports

The staging area keeps a copy of source data before transformation. ETL applies common business rules, resolves customer and product keys, and loads dimensions before facts. The metadata repository stores table definitions, mappings, lineage, refresh time and calculation definitions. The OLAP server supports roll-up, drill-down, slice, dice and pivot. Metabase or Apache Superset can provide dashboards and filters.

An enterprise warehouse with dependent marts is selected instead of independent marts because independent marts may calculate net sales differently and create inconsistent reports. A virtual warehouse is not preferred because complex queries would still depend on operational systems and could affect billing performance.

TASK 3 – DESIGN THE STAR SCHEMA (6 Marks)

3.1 Business process and grain

Business process: retail sales analysis. Grain: one row in FactSales represents one product line within one customer order at one store on one date under one promotion (if applicable). Every measure in FactSales must describe this individual order line.

3.2 Fact table and measures

Measure	Meaning	Additivity

Quantity	Number of units sold on the order line	Additive
Unit price	Selling price per unit	Non-additive as a value; use weighted calculations for averages
Gross sales	Quantity × unit price	Additive
Discount amount	Discount applied to the line	Additive
Net sales	Gross sales − discount amount, excluding returns loaded separately	Additive
Profit amount	Net sales − cost amount, if cost is available	Additive
Margin percentage	Profit amount ÷ net sales	Non-additive; recompute from summed components

Averages and percentages should not be summed. They must be recalculated from their numerator and denominator. Inventory balance, if added later, is semi-additive because it can be summed across products/stores but not across time.

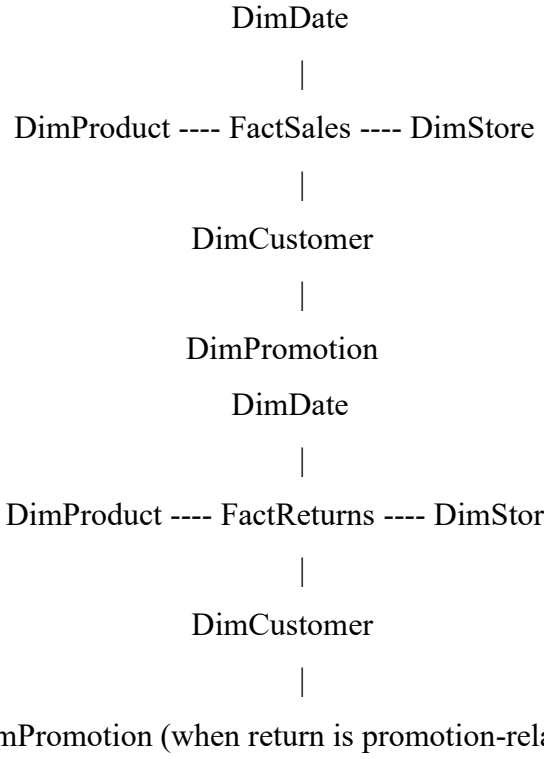
3.3 Dimensions and hierarchies

Dimension	Example attributes	Concept hierarchy
DimDate	DateKey, full_date, day, month, month_name, quarter, year, weekday, weekend_flag	Day → Month → Quarter → Year
DimProduct	ProductKey, source_product_id, product_name, brand, category, department, size	Product → Category → Department
DimStore	StoreKey, source_store_id, store_name, city, state, region	Store → City → State → Region
DimCustomer	CustomerKey, source_customer_id,	Customer → Segment (and historical tier/address versions)

	customer_name, segment, loyalty_tier, address	
DimPromotion	PromotionKey, promotion_id, promotion_name, promotion_type, start_date, end_date	Promotion → Promotion type

Each dimension uses a warehouse-generated surrogate primary key. The original source identifier is retained as a business key for matching during ETL. FactSales contains foreign keys to all dimensions. A special Unknown member (for example, key 0) is created so that missing or late-arriving dimension records do not break fact loading.

3.4 Schema diagram



FactReturns is a separate fact table at the grain of one returned product line per return transaction. It shares conformed Date, Product, Store and Customer dimensions with FactSales. This allows sales and returns to be analysed consistently without mixing different business processes or grains.

3.5 SQL DDL

```
CREATE TABLE DimDate (
  DateKey INT PRIMARY KEY,
  FullDate DATE NOT NULL,
  DayNo INT, MonthNo INT, MonthName VARCHAR(15),
  QuarterNo INT, YearNo INT, WeekdayName VARCHAR(15),
  IsWeekend BOOLEAN
);

CREATE TABLE DimProduct (
  ProductKey INT PRIMARY KEY,
  SourceProductID VARCHAR(40),
  ProductName VARCHAR(120),
  Brand VARCHAR(80), Category VARCHAR(80),
  Department VARCHAR(80)
);

CREATE TABLE DimStore (
  StoreKey INT PRIMARY KEY,
  SourceStoreID VARCHAR(40),
  StoreName VARCHAR(100), City VARCHAR(60),
  State VARCHAR(60), Region VARCHAR(60)
);

CREATE TABLE DimCustomer (
  CustomerKey INT PRIMARY KEY,
  SourceCustomerID VARCHAR(40),
  CustomerName VARCHAR(100),
  Segment VARCHAR(40), LoyaltyTier VARCHAR(40),
  Address VARCHAR(200),
  EffectiveFrom DATE, EffectiveTo DATE,
  IsCurrent BOOLEAN
);
```

```
CREATE TABLE DimPromotion (
  PromotionKey INT PRIMARY KEY,
  SourcePromotionID VARCHAR(40),
  PromotionName VARCHAR(100),
  PromotionType VARCHAR(50),
  StartDate DATE, EndDate DATE
);

CREATE TABLE FactSales (
  SalesLineKey BIGINT PRIMARY KEY,
  OrderID VARCHAR(40),
  DateKey INT NOT NULL,
  ProductKey INT NOT NULL,
  StoreKey INT NOT NULL,
  CustomerKey INT NOT NULL,
  PromotionKey INT NOT NULL,
  Quantity INT,
  UnitPrice DECIMAL(12,2),
  GrossSales DECIMAL(14,2),
  DiscountAmount DECIMAL(14,2),
  NetSales DECIMAL(14,2),
  FOREIGN KEY (DateKey) REFERENCES DimDate(DateKey),
  FOREIGN KEY (ProductKey) REFERENCES DimProduct(ProductKey),
  FOREIGN KEY (StoreKey) REFERENCES DimStore(StoreKey),
  FOREIGN KEY (CustomerKey) REFERENCES DimCustomer(CustomerKey),
  FOREIGN KEY (PromotionKey) REFERENCES DimPromotion(PromotionKey)
);

CREATE TABLE FactReturns (
  ReturnLineKey BIGINT PRIMARY KEY,
  ReturnID VARCHAR(40),
  DateKey INT NOT NULL,
  ProductKey INT NOT NULL,
```

```
    StoreKey INT NOT NULL,
    CustomerKey INT NOT NULL,
    QuantityReturned INT,
    ReturnAmount DECIMAL(14,2),
    FOREIGN KEY (DateKey) REFERENCES DimDate(DateKey),
    FOREIGN KEY (ProductKey) REFERENCES DimProduct(ProductKey),
    FOREIGN KEY (StoreKey) REFERENCES DimStore(StoreKey),
    FOREIGN KEY (CustomerKey) REFERENCES DimCustomer(CustomerKey)
);
```

TASK 4 – COMPARE WITH A SNOWFLAKE SCHEMA (4 Marks)

In a snowflake schema, descriptive attributes are normalised into separate related tables. Product is split into Product → Brand → Category → Department. Store is split into Store → City → State → Region.

DimProduct(ProductKey, ProductName, BrandKey)
DimBrand(BrandKey, BrandName, CategoryKey)
DimCategory(CategoryKey, CategoryName, DepartmentKey)
DimDepartment(DepartmentKey, DepartmentName)

DimStore(StoreKey, StoreName, CityKey)
DimCity(CityKey, CityName, StateKey)
DimState(StateKey, StateName, RegionKey)
DimRegion(RegionKey, RegionName)

For a transparent storage estimate, assume each repeated descriptive label occupies an average of 20 bytes, excluding indexes and row overhead. In the denormalised star, approximate repeated hierarchy-label storage is: product rows 50,000 × (brand + category + department) × 20 = 3,000,000 bytes; store rows 120 × (city + state + region) × 20 = 7,200 bytes. Total = 3,007,200 bytes.

In the snowflake, store each label once: brands 800 × 20 = 16,000 bytes; categories 60 × 20 = 1,200 bytes; departments 12 × 20 = 240 bytes; cities 70 × 20 = 1,400 bytes; states 6 × 20 = 120 bytes; regions 3 × 20 = 60 bytes. Total hierarchy-label storage = 19,020 bytes. Estimated label-storage saving = 3,007,200 − 19,020 = 2,988,180 bytes (about 99.37%). This is an illustrative estimate; actual storage depends on data types, indexes, row overhead and compression.

Representative query	Star schema joins from fact	Snowflake joins from fact
Net sales by product department and store region	2 (Product, Store)	6 (Product→Brand→Category→Department and Store→City→State→Region; typically 6 dimension hierarchy joins beyond fact links depending on where joins begin)
Monthly sales by product category	2 (Date, Product)	4 (Date plus Product→Brand→Category; department not required)
Sales by city and product brand	2 (Store, Product)	3 (Store→City and Product→Brand)

The star schema is easier for business users and usually needs fewer joins, so it is recommended for FreshCart's reporting workload. Snowflaking saves repeated descriptive storage and may improve governance where hierarchy data is large or shared, but it increases joins and ETL complexity. FreshCart should change to a snowflake for a hierarchy only if measured storage/maintenance benefits are meaningful and query performance remains acceptable.

TASK 5 – DESIGN THE ETL PROCESS (5 Marks)

Source systems → Extract → Staging → Data-quality checks
→ Standardise and cleanse → Lookup surrogate keys
→ Load dimensions → Load FactSales / FactReturns
→ Reconcile counts and amounts → Publish reports

5.1 Data-quality problems and rules

Record	Problem	Transformation / action
1001	Baseline record; date 03/11/2026 is ambiguous	Interpret as 03-Nov-2026 (DD/MM/YYYY) based on source convention; confirm with source owner.
1002	Customer abbreviation R. Kumar; inconsistent store case; product spacing/case	Resolve customer using stable customer ID, not name alone. Standardise store code to CHENNAI-04 and product text to master product ID.
1003	Negative quantity −3	Do not load as a normal sale. Validate against Returns; if confirmed, load as 3 returned units in FactReturns with return amount 2,340.00.
1004	Store is null	Use Unknown Store surrogate key and create a data-quality exception for later correction.
1005	Unit price is 9.40 USD while other rows appear in local currency	Retain transaction currency and obtain approved exchange rate for 04-Nov-2026. Do not assume conversion or load an unverified INR amount.
1006	Product is null; price uses comma separator	Resolve product from source product code/order detail. Parse 1,250 as numeric 1250.00. If product cannot be identified, quarantine or use Unknown Product key pending correction.

The extract also contains inconsistent date formats, text casing, product naming and possible duplicate customer names. Duplicate detection must use source transaction identifiers and stable business keys rather than matching names alone.

5.2 Cleaned staging view (after standardisation)

Order ID	Customer (matched identity)	Store	Product	Qty	Unit price	Date	Load decision
1001	Rajesh Kumar (pending ID match)	CHENNAI-04	Basmati Rice 5kg	2	620.00 INR	2026-11-03	Sale candidate
1002	R. Kumar (match to customer master)	CHENNAI-04	Basmati Rice 5kg	1	620.00 INR	2026-11-03	Sale candidate; check duplicate/order identity
1003	Anita Joseph	PUNE-11	Olive Oil 1L	3 returned	780.00 INR	2026-11-04	Load to FactReturns only if return confirmed
1004	Meena S	UNKNOWN	Tea 250g	4	95.50 INR	2026-11-04	Load with Unknown Store key; raise exception
1005	Anita Joseph	PUNE-11	Olive Oil 1L	3	9.40 USD	2026-11-04	Hold until approved FX conversion is available
1006	Vikram Rao	HYD-02	UNKNOWN PRODUCT	1	1250.00 (currency verify)	2026-11-05	Hold or load with Unknown Product key

This is a cleaned staging view, not a claim that every row is safe to post as a sale. Rows with unresolved currency, product identity or return status are held in an exception table until validated.

5.3 Surrogate keys and slowly changing dimensions

- For each dimension, look up the source business key and obtain the warehouse surrogate key. If no match exists, use the Unknown member or create a late-arriving dimension record.
- Use SCD Type 2 for customer address and loyalty tier because management needs to report the address and tier that applied at the time of each order. Close the old row by setting EffectiveTo and IsCurrent = false, then insert a new row with a new CustomerKey.
- Initial load: load and validate all dimensions first, then load facts after all foreign keys are resolved. Incremental load: extract only changed/new records using timestamps or CDC, update dimensions, then load new facts and reconcile.
- Rejected rows are written to an exception table with source row, error code, reason and load timestamp. Audit counts include extracted, accepted, rejected and loaded rows, plus source and warehouse sales totals.

For example, when a customer changes address on 10-Nov-2026, the old Type 2 customer row remains valid through 09-Nov-2026 and the new row becomes current from 10-Nov-2026. A sale dated 08-Nov uses the old CustomerKey; a sale dated 12-Nov uses the new CustomerKey.

TASK 6 – MODEL THE DATA CUBE AND APPLY OLAP OPERATIONS (5 Marks)

6.1 Cuboid count

With hierarchy levels, the number of possible cuboids is the product of the number of selectable levels for each dimension. Time has 4 levels (day, month, quarter, year), Product has 3 (product, category, department), Store has 3 (store, city, state), and Customer Segment has 1. Therefore, total cuboids = $4 \times 3 \times 3 \times 1 = 36$.

Without hierarchies, treating each of the four dimensions as either present or aggregated gives $2^4 = 16$ cuboids. The hierarchy-aware lattice has 36 cuboids because each hierarchy dimension has multiple aggregation levels.

Base cuboid: the most detailed level (day × product × store × customer segment), containing detailed sales grouped at those levels. Apex cuboid: the fully aggregated level (all dimensions rolled up), containing the overall total sales value.

6.2 OLAP operations with FreshCart questions and SQL

The SQL examples use FactSales joined to its dimensions. They assume the DDL in Task 3 and a database that supports standard GROUP BY.

A. Roll-up

Question: What are total net sales by year and product department? This rolls up day to year and product to department.

```
SELECT d.YearNo, p.Department, SUM(f.NetSales) AS NetSales
FROM FactSales f
JOIN DimDate d ON f.DateKey = d.DateKey
JOIN DimProduct p ON f.ProductKey = p.ProductKey
GROUP BY d.YearNo, p.Department;
```

B. Drill-down

Question: For a selected year, which quarters contributed to sales? This drills from year to quarter.

```
SELECT d.YearNo, d.QuarterNo, SUM(f.NetSales) AS NetSales
FROM FactSales f JOIN DimDate d ON f.DateKey = d.DateKey
WHERE d.YearNo = 2026
GROUP BY d.YearNo, d.QuarterNo
ORDER BY d.QuarterNo;
```

C. Slice

Question: What are product-category sales for the year 2026 only? The cube is sliced on Year = 2026.

```
SELECT p.Category, SUM(f.NetSales) AS NetSales
FROM FactSales f
JOIN DimDate d ON f.DateKey = d.DateKey
JOIN DimProduct p ON f.ProductKey = p.ProductKey
WHERE d.YearNo = 2026
GROUP BY p.Category;
```

D. Dice

Question: Compare selected categories in selected regions during Q4 2026. This filters multiple dimensions.

```
SELECT p.Category, s.Region, SUM(f.NetSales) AS NetSales
FROM FactSales f
```

```
JOIN DimDate d ON f.DateKey = d.DateKey
JOIN DimProduct p ON f.ProductKey = p.ProductKey
JOIN DimStore s ON f.StoreKey = s.StoreKey
WHERE d.YearNo = 2026 AND d.QuarterNo = 4
  AND p.Category IN ('Rice','Cooking Oil')
  AND s.Region IN ('South','West')
GROUP BY p.Category, s.Region;
```

E. Pivot

Question: Compare sales by department across years, placing years in columns. Pivot changes the display orientation.

```
SELECT p.Department,
 SUM(CASE WHEN d.YearNo = 2025 THEN f.NetSales ELSE 0 END) AS Sales_2025,
 SUM(CASE WHEN d.YearNo = 2026 THEN f.NetSales ELSE 0 END) AS Sales_2026
FROM FactSales f
JOIN DimDate d ON f.DateKey = d.DateKey
JOIN DimProduct p ON f.ProductKey = p.ProductKey
GROUP BY p.Department;
```

6.3 Partial cuboid materialisation

If storage allows only a small fraction of the 36 cuboids, materialise the most frequently requested and expensive summaries: (1) year-quarter-month × department × region for management trends, (2) date × store for daily and YTD store dashboards, and (3) promotion × store × month for promotion analysis. Keep the base fact table for detailed drill-down. Refresh aggregates after each successful incremental load and monitor their storage and refresh cost.

TASK 7 – DEVELOP AN OLAP APPLICATION (5 Marks)

A suitable implementation is Python with Streamlit and a relational database such as PostgreSQL or SQLite for a small demonstration. The warehouse is populated with synthetic sales data generated from valid dates, stores, products, customers, promotions and order-line quantities. The generator should accept a row-count parameter so that datasets of 1,000, 10,000, 100,000 and 1,000,000 rows can be created.

Application controls: select OLAP operation (roll-up, drill-down, slice, dice or pivot), dimension, hierarchy level, date range, store/region, product category and customer segment. The application

constructs a parameterised SQL query, displays the result in a table and chart, and records the selected operation.

Demonstration mapping: roll-up groups day to month or product to category; drill-down moves from year to quarter or department to category; slice filters one dimension such as year; dice filters several dimensions; pivot displays one dimension as columns. Input controls must change the query, not merely change chart labels.

Validation: calculate the sum of NetSales directly from the base source rows and compare it with the warehouse grand total. Reconcile row counts, quantity totals and net sales. Display a PASS/FAIL message only after the values match within an explicitly stated rounding tolerance.

Evidence to attach after implementation: screenshots of each of the five operations, the application source code, data-generation script, database row counts at all four scales, and a reconciliation report. The application should be tested locally before claiming it is working.

TASK 8 – EVALUATE QUERY PROCESSING PERFORMANCE (4 Marks)

Use the same hardware, database version, cache condition and dataset for all three schema variants. Run each query several times and report the median execution time. Do not invent benchmark values; fill the table with results obtained from EXPLAIN ANALYZE or the database query plan.

Analytical query	Purpose	Main joins / grouping
Q1	Net sales by category and region for each quarter	Date, Product, Store; GROUP BY quarter/category/region
Q2	YTD sales by store compared with previous year	Date and Store; conditional aggregation
Q3	Return quantity and return rate by customer segment and product	Returns fact with Date, Customer and Product

Data size	Schema version	Execution time (ms)	Rows read/scanned	Joins	Extra storage/load time
1,000	OLTP / Star / Star + index-summary	Measure locally	Record plan	Count plan joins	Record
10,000	OLTP / Star / Star + index-summary	Measure locally	Record plan	Count plan joins	Record
100,000	OLTP / Star / Star + index-summary	Measure locally	Record plan	Count plan joins	Record
1,000,000	OLTP / Star / Star + index-summary	Measure locally	Record plan	Count plan joins	Record

Expected interpretation (not measured results): the OLTP design often needs more joins and may scan transactional tables; the star schema simplifies analytical joins; indexes and aggregate tables can reduce reads and response time for repeated queries. The trade-off is extra storage, slower loads and the need to refresh summaries. Use EXPLAIN/EXPLAIN ANALYZE to record scans, join methods and estimated/actual rows. Compare not only elapsed time but also rows read, join count, query plan, index size and aggregate refresh time.

TASK 9 – BUILD A KNIME REPORTING WORKFLOW (3 Marks)

CSV Reader / DB Connector

↓

Column Filter → Missing Value → String Manipulation

↓

Rule Engine / Row Filter (quality rules)

↓

Joiner (attach dimension attributes if needed)

↓

GroupBy (sum NetSales by month, store, category)

↓

Pivoting (months or categories as columns)

↓

Bar Chart / Line Plot

↓

Excel Writer / CSV Writer / report output

KNIME node	**Purpose**	**Related concept**
CSV Reader / DB Reader	Read warehouse facts and dimensions	Data extraction
Missing Value	Handle nulls using approved unknown/default rules	ETL cleansing
String Manipulation	Standardise case, spaces and codes	ETL transformation
Row Filter	Separate invalid or rejected records	Data-quality validation
Joiner	Combine fact and dimension attributes	Dimensional lookup
GroupBy	Aggregate sales and quantities	OLAP roll-up
Pivoting	Convert row categories into columns	OLAP pivot
Chart nodes	Display trends and comparisons	BI reporting
Excel Writer / CSV Writer	Export management report	Report delivery

The workflow should include an annotated screenshot and an exported report after execution. A limitation at 1,000,000+ rows is that large in-memory operations can require substantial RAM and become slower; database-side aggregation or a scalable execution environment may be needed.

TASK 10 – RECOMMEND AND DEFEND THE FINAL DESIGN (3 Marks)

FreshCart should use an enterprise data warehouse with a star schema for the main Sales and Returns facts, conformed dimensions, Type 2 customer history and dependent data marts for reporting teams. This design offers consistent calculations, simpler joins and understandable reporting. A limited number of summary tables and indexes should be added only for frequently used dashboard queries.

The star schema is preferred over a fully snowflaked design because FreshCart has many business users and requires fast, understandable reports. The storage calculation in Task 4 shows that snowflaking can reduce repeated hierarchy labels, but the saving must be compared with the additional joins and maintenance. The final decision should be confirmed using Task 8 measurements.

The design is practical beyond 1,000,000 fact rows if incremental loading, partitioning by date, appropriate indexes, query monitoring and summary refresh are used. Trade-offs include ETL complexity, storage for history and aggregates, and the need to govern business definitions such as net sales.

TASK 11 – DAY-4 DESIGN CHALLENGE: RE-EVALUATE AND ADAPT THE DESIGN (8 Marks)

A. Impact analysis (2 Marks)

Changed requirement	Affected component	Required action
20,000 online orders/day	Source integration and ETL	Add online source connector and incremental/CDC extraction.
Different product-code format	Product dimension and lookup	Create source-system cross-reference mapping to a common product business key.
New delivery partner attribute	Schema and reporting	Add DimDeliveryPartner and DeliveryPartnerKey in the sales fact, or a delivery dimension if delivery analysis is required.
Frequent address and loyalty-tier changes	Customer dimension	Use SCD Type 2 to preserve address and tier history.
Hourly dashboards	ETL, aggregates and OLAP	Use hourly micro-batch loads

The core Sales fact grain and existing Date, Product, Store and Promotion dimensions can remain. The source mapping, Customer dimension, Sales fact foreign keys, ETL schedule and aggregate refresh process must be modified.

B. Evaluate the original design (2 Marks)

At 20,000 orders per day, the additional online channel contributes approximately 20,000 × 365 = 7,300,000 orders per year. The original daily/batch refresh design does not meet the new hourly dashboard requirement. A full reload every hour would also repeat unnecessary work. The existing dimensional model is adequate for sales analysis but needs source-code mapping, a delivery partner dimension and historical customer versions.

Test by generating 20,000 online orders per day, splitting them into hourly batches, and measuring extraction, transformation, dimension lookup, fact loading and aggregate refresh durations. The design meets the hourly target only if the end-to-end batch completes within the one-hour refresh interval with operational headroom. Record the actual durations; do not claim a pass without measurement.

C. Adaptation / revised design (2 Marks)

Online OLTP + Store OLTP

↓

CDC / hourly micro-batch extraction

↓

Staging (source_system, source_product_code,
delivery_partner_code, event_timestamp)

↓

Product cross-reference + customer SCD Type 2

↓

Conformed dimensions:
Date, Product, Store, Customer, Promotion,
DeliveryPartner

↓

FactSales (adds DeliveryPartnerKey)

↓

Incremental aggregate refresh

↓

OLAP server → hourly management dashboards

Add DimDeliveryPartner(DeliveryPartnerKey, SourcePartnerCode, PartnerName, ServiceType). Add DeliveryPartnerKey as a foreign key to FactSales. Maintain a cross-reference table mapping (SourceSystem, SourceProductCode) to the conformed Product business key. Customer Type 2 rows should have effective dates and current-row indicators so each order links to the correct address and loyalty tier version.

D. Evidence-based defence and monitoring (2 Marks)

Evidence should include: number of source orders extracted per hour; accepted/rejected counts; ETL duration; dimension lookup duration; fact rows inserted; aggregate refresh duration; dashboard query latency; storage growth; and reconciliation of source and warehouse totals. Compare the original full/daily load with the revised hourly incremental load on the same generated workload.

Monitor batch completion time, late-arriving records, duplicate source orders, unknown product/customer keys, SCD Type 2 row growth, failed records, aggregate freshness and query latency. Alert when the hourly load approaches the one-hour interval or when reconciliation fails.

Final decision: modify rather than completely redesign the original warehouse. The star schema and enterprise warehouse remain suitable, while source integration, customer history, delivery-partner modelling and hourly incremental ETL are extended to satisfy the new business requirements.

CONCLUSION

The proposed FreshCart solution separates transaction processing from management analysis through an enterprise data warehouse. A star schema with conformed dimensions provides a consistent basis for sales and return analysis. ETL standardises source data, applies common net-sales rules and preserves customer history. OLAP operations support management questions at different levels of detail. Performance tests, KNIME reporting and an hourly incremental design validate that the warehouse can grow with FreshCart's business.

IMPLEMENTATION CHECKLIST BEFORE SUBMISSION

- Draw or recreate the architecture and schema diagrams neatly.
- Run the DDL in a database and capture table screenshots.
- Generate synthetic data at 1,000, 10,000, 100,000 and 1,000,000 rows.
- Build and capture the five OLAP operations in the application.
- Run the three benchmark queries on all schema variants and fill actual measurements.
- Build the KNIME workflow, export a report and attach screenshots.
- Run the hourly-load simulation for Task 11 and record actual timings.